

UNIVERSITY OF WASHINGTON

TINY MACHINE LEARNING FOR ULTRA LOW-POWER EDGE COMPUTING

EE P 564 A

STOP & LOG:
A Voice-Triggered Distraction Tracker

Authors: Eric Peng, William Liao

May 22, 2026



Mid-Project Checkpoint Report

Project Status and Milestone Review

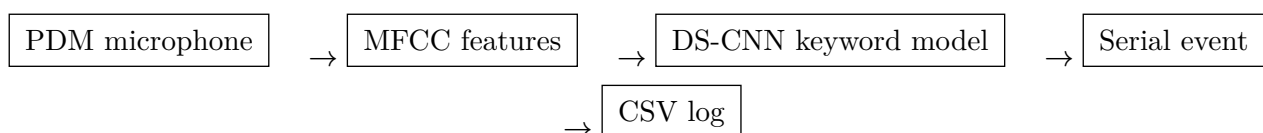
Our project, *STOP & LOG*, is a TinyML keyword-triggered distraction tracker. The intended workflow is to detect the spoken keyword “STOP” on an Arduino Nano 33 BLE Sense, then send a detection event over Serial so a computer-side Python script can append the event to a CSV distraction log.

The original proposal defined three main milestones: (1) prepare the Speech Commands dataset and validate the Lab 4 audio pipeline, (2) retrain and quantize a “stop” versus “background/unknown” keyword model for Arduino deployment, and (3) complete the end-to-end logging path and evaluate the system in realistic use. At this checkpoint, the dataset, model architecture, deployment target, and runtime logging architecture have been finalized. The PC-side logging component has also been implemented as a reusable Python script. The model training and on-board validation milestone is still in progress and is the primary focus for the remaining phase.

Milestone	Status	Checkpoint notes
Dataset and preprocessing plan	Met	Google Speech Commands v0.02 remains the primary dataset. The target class is “stop”; all other words plus background noise form the negative class. MFCC features from 1-second, 16 kHz audio clips match the proposed TinyML audio pipeline.
Model design	In progress	The planned model is still a compact DS-CNN with post-training int8 quantization. The next step is to finish training the binary classifier and compare model size, accuracy, and false positives before Arduino export.
Deployment architecture	Partially met	The Arduino Nano 33 BLE Sense remains the target. The runtime interface has been specified as one Serial event per accepted keyword detection, which keeps the embedded side simple and makes the logger easy to test.
Logging and analysis	Met	A Python Serial logger has been prepared to write timestamped detections to CSV. It records ISO timestamp, date, time, day of week, event label, confidence when available, raw Serial line, and port.
Real-world evaluation	Pending	Evaluation will be run after the quantized model is deployed. Planned tests include quiet-room detection, ambient-noise false positives, similar-word rejection, and end-to-end latency.

Current Architecture

The system is organized as a small streaming pipeline:



On the embedded side, the Arduino continuously captures 1-second audio windows from the built-in microphone, extracts MFCC features, and classifies each window as either “stop” or “background/unknown.” To reduce accidental logs, the final implementation will use a confidence threshold and may require repeated high-confidence detections before emitting an event. On the host side, the Python script treats each accepted Serial line as a detection event and records it with the computer’s local timestamp.

Accomplishments and Preliminary Conclusions

The main checkpoint accomplishment is that the project scope has become more concrete and testable. The binary keyword formulation is simpler than a full command recognizer, which should help reduce model size and make evaluation easier. The Google Speech Commands dataset already contains the target word “stop” and a large pool of negative examples, so the project does not depend on collecting a large custom dataset before training can begin.

The second accomplishment is separating the system into two independently testable parts. The Serial logger can be tested before the TinyML model is finished by sending synthetic lines such as `STOP,0.92` or `stop detected confidence=0.87`. This lowers integration risk because logging, timestamp formatting, CSV persistence, and data analysis do not depend on the Arduino model being perfect.

The preliminary conclusion is that the project remains feasible for the final deadline, but the most important unknowns are now model quality and false positive behavior in realistic environments. We should avoid claiming success from dataset accuracy alone; a useful distraction tracker must be conservative enough that ordinary speech, typing noise, and similar-sounding words do not create frequent false logs.

Bottlenecks and Mitigation Plan

The largest bottleneck is false positives from non-target audio. The mitigation is to train with diverse negative examples, reserve a realistic validation set, tune the detection threshold, and test similar words such as “shop,” “top,” and “stopwatch.” If needed, the Arduino logic will require two consecutive high-confidence windows before sending a log event.

The second bottleneck is mismatch between public dataset audio and the actual user’s environment. To address this, we plan to add a small supplementary dataset of personal “stop” recordings and common local background sounds. This should improve robustness without changing the overall training pipeline.

The third bottleneck is deployment size. The mitigation remains post-training int8 quantization and a compact DS-CNN architecture. Before replacing the Arduino `micro_features_model.cpp`, the exported model size will be checked against the Nano 33 BLE Sense memory budget.

Remaining Work

The remaining work is to complete training, quantization, and deployment, then run a focused evaluation. The final evaluation will report held-out test accuracy, approximate model size, observed false positives per minute under common ambient conditions, and end-to-end latency from saying “STOP” to a CSV row appearing on the computer. If time allows, we will also compare performance with and without custom voice recordings to determine whether personalization meaningfully improves the tracker.